

Java Output / Print

Mastering display and formatting in Java

1. The Primary Methods

Java's standard way of talking back to the user is through the **System** class. The two workhorse methods are `println()` and `print()`.

System.out.println()

Prints the text and **jumps to a new line**. Most common for lists or separate messages.

System.out.print()

Prints the text but **stays on the same line**. Useful for building sentences piece by piece.

```
// Visualizing the difference:  
System.out.println("Line 1");  
System.out.print("Part A ");  
System.out.print("Part B");
```

Output:
Line 1
Part A Part B

2. Handling Values

While Text (Strings) requires double quotes, numbers and booleans do not.

```
System.out.println("Age: " + 25); // Combining text and number  
System.out.println(3.14159); // Large decimal  
System.out.println(true); // Boolean state
```

3. String Concatenation

The + symbol is used for **Concatenation**—which is a fancy word for joining things together into one string.

Example:

```
String name = "Alice";  
System.out.println("Welcome back, " + name + "!");  
→ Output: Welcome back, Alice!
```

4. Escape Characters

Sometimes you need to print things that have special meaning in Java, like quotes or new lines. We use the backslash \ as an escape signal.

Code	Description	Sample Usage
	New Line	"Hello World"
	Tab Space	"Name: John"
"	Double Quote	"Say \"Hi\""
\	Backslash	"C:\\Folder"

⚠ CRITICAL RULE: Forget the double quotes around text, and Java will think you are calling a variable that doesn't exist, causing a crash!

5. Quick Recap

println = New Line

print = Same Line

"" = For Text

;; = Required Line Ender

+ = Join Together

\\ = Next Line Escape

Did you know?

Printing results is the most common way to **debug** (find errors) in your code. If a variable isn't behaving, print it out to see what happened!

Codehive